```
import pandas as pd

data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 2, 6],
    "Attendance": [50, 55, 60, 65, 70, 80, 85, 90, 60, 75],
    "Previous_Marks": [35, 40, 45, 50, 55, 65, 70, 80, 42, 68],
    "Final_Marks": [30, 35, 42, 48, 55, 65, 72, 85, 38, 67]
}

df = pd.DataFrame(data)

print(df)
```

```
   Study_Hours  Attendance  Previous_Marks  Final_Marks
0            1          50              35           30
1            2          55              40           35
2            3          60              45           42
3            4          65              50           48
4            5          70              55           55
5            6          80              65           65
6            7          85              70           72
7            8          90              80           85
8            2          60              42           38
9            6          75              68           67
```

```
# Input features: information used to make predictions
X = df[["Study_Hours", "Attendance", "Previous_Marks"]]

# Target: the marks we want the model to predict
y = df["Final_Marks"]

print("Input features (X):")
print(X.head())

print("\nTarget values (y):")
print(y.head())
```

```
Input features (X):
   Study_Hours  Attendance  Previous_Marks
0            1          50              35
1            2          55              40
2            3          60              45
3            4          65              50
4            5          70              55

Target values (y):
0    30
1    35
2    42
3    48
4    55
Name: Final_Marks, dtype: int64
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training input rows:", len(X_train))
print("Testing input rows:", len(X_test))

print("Training target rows:", len(y_train))
print("Testing target rows:", len(y_test))
```

```
Training input rows: 8
Testing input rows: 2
Training target rows: 8
Testing target rows: 2
```

```
from sklearn.linear_model import LinearRegression

# Create the machine learning model
model = LinearRegression()

# Train the model using our training data
model.fit(X_train, y_train)

print("Model training completed successfully!")
```

```
Model training completed successfully!
```

```
# Predict final marks for students in the test set
predictions = model.predict(X_test)

print("Actual marks:", list(y_test))
print("Predicted marks:", predictions.round(2))
```

```
Actual marks: [38, 35]
Predicted marks: [38.61 35.73]
```

```
from sklearn.metrics import mean_absolute_error

# Calculate prediction error
mae = mean_absolute_error(y_test, predictions)

print("Mean Absolute Error:", round(mae, 2))
```

```
Mean Absolute Error: 0.67
```

```
# Enter details for a new student
study_hours = 5
attendance = 80
previous_marks = 65

# Prepare the input in the same format used during training
new_student = pd.DataFrame({
    "Study_Hours": [study_hours],
    "Attendance": [attendance],
    "Previous_Marks": [previous_marks]
})

# Predict the student's final marks
predicted_marks = model.predict(new_student)

print("Student Details")
print("Study hours:", study_hours)
print("Attendance:", attendance, "%")
print("Previous marks:", previous_marks)

print("Predicted final marks:", round(predicted_marks[0], 2))
```

```
Student Details
Study hours: 5
Attendance: 80 %
Previous marks: 65
Predicted final marks: 65.72
```

```
# Compare actual marks with predicted marks
comparison = pd.DataFrame({
    "Actual_Marks": y_test.values,
    "Predicted_Marks": predictions.round(2)
})

comparison["Absolute_Error"] = (
    comparison["Actual_Marks"] - comparison["Predicted_Marks"]
).abs().round(2)

print("Actual vs Predicted Marks")
display(comparison)
```

```
Actual vs Predicted Marks
```

	Actual_Marks	Predicted_Marks	Absolute_Error
0	38	38.61	0.61
1	35	35.73	0.73

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

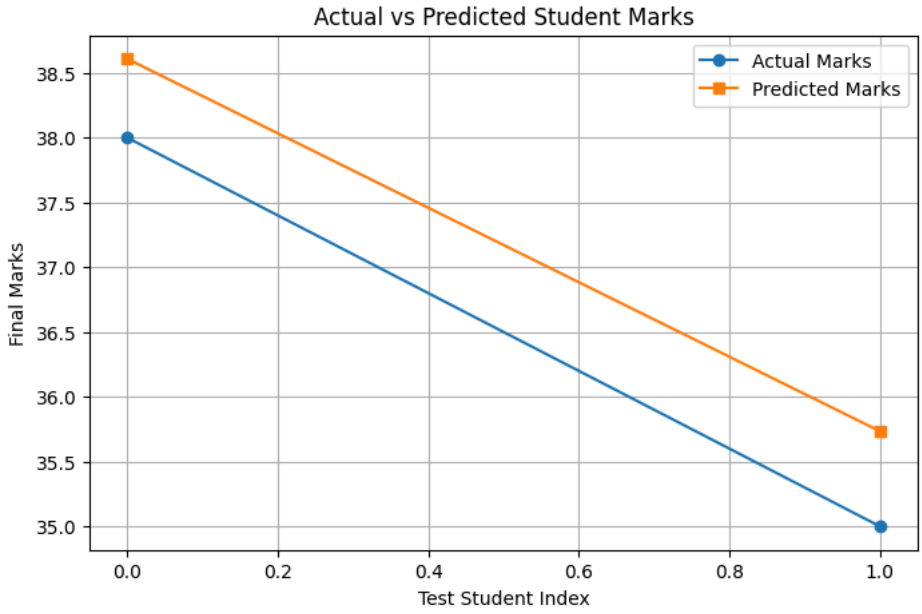
plt.plot(
    range(len(y_test)),
    y_test.values,
    marker="o",
    label="Actual Marks"
)

plt.plot(
    range(len(predictions)),
    predictions,
    marker="s"
```

```
    label="Predicted Marks"
)

plt.xlabel("Test Student Index")
plt.ylabel("Final Marks")
plt.title("Actual vs Predicted Student Marks")
plt.legend()
plt.grid(True)

plt.show()
```




```
import joblib

# Save the trained model to a file
joblib.dump(model, "student_performance_model.pkl")

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
# Load the saved model
loaded_model = joblib.load("student_performance_model.pkl")

# Predict marks using the loaded model
test_prediction = loaded_model.predict(new_student)

print("Prediction using saved model:", round(test_prediction[0], 2))
```

```
Prediction using saved model: 65.72
```

```
def predict_student_marks(study_hours, attendance, previous_marks):
    student = pd.DataFrame({
        "Study_Hours": [study_hours],
        "Attendance": [attendance],
        "Previous_Marks": [previous_marks]
    })

    predicted_marks = loaded_model.predict(student)[0]

    # Keep the result within the usual 0-100 marks range
    predicted_marks = max(0, min(100, predicted_marks))

    return round(predicted_marks, 2)


# Try the function with one student
result = predict_student_marks(5, 80, 65)

print("Predicted final marks:", result)
```

```
Predicted final marks: 65.72
```

```
def predict_student_marks(study_hours, attendance, previous_marks):
    student = pd.DataFrame({
        "Study_Hours": [study_hours],
```

```
        "Attendance": [attendance],
        "Previous_Marks": [previous_marks]
    })

    predicted_marks = loaded_model.predict(student)[0]

    predicted_marks = max(0, min(100, predicted_marks))

    return round(predicted_marks, 2)

print("Prediction function is ready!")
```

```
Prediction function is ready!
```

```
print("Pandas loaded:", "pd" in globals())
print("Saved model loaded:", "loaded_model" in globals())
print("Prediction function defined:", "predict_student_marks" in globals())
```

```
Pandas loaded: False
Saved model loaded: False
Prediction function defined: True
```

Start coding or generate with AI.

```
print("Original model available:", "model" in globals())
print("Training data available:", "X_train" in globals())
print("Target data available:", "y_train" in globals())
print("Pandas available:", "pd" in globals())
```

```
Original model available: False
Training data available: False
Target data available: False
Pandas available: True
```

```
import pandas as pd

data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 2, 6],
    "Attendance": [50, 55, 60, 65, 70, 80, 85, 90, 60, 75],
    "Previous_Marks": [35, 40, 45, 50, 55, 65, 70, 80, 42, 68],
    "Final_Marks": [30, 35, 42, 48, 55, 65, 72, 85, 38, 67]
}

df = pd.DataFrame(data)

print("Dataset restored successfully!")
display(df)
```

```
Dataset restored successfully!
```

	Study_Hours	Attendance	Previous_Marks	Final_Marks
0	1	50	35	30
1	2	55	40	35
2	3	60	45	42
3	4	65	50	48
4	5	70	55	55
5	6	80	65	65
6	7	85	70	72
7	8	90	80	85
8	2	60	42	38
9	6	75	68	67

Next steps: Generate code with df | New interactive sheet

```
X = df[["Study_Hours", "Attendance", "Previous_Marks"]]
y = df["Final_Marks"]

print("Input data (X):")
display(X.head())

print("Output data (y):")
display(y.head())
```

```
Input data (X):
```

	Study_Hours	Attendance	Previous_Marks
0	1	50	35
1	2	55	40
2	3	60	45
3	4	65	50
4	5	70	55

```
Output data (y):
```

	Final_Marks
0	30
1	35
2	42
3	48
4	55

dtype: int64

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 8
Testing samples: 2
```

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

```
predictions = model.predict(X_test)

print("Predicted marks:")
print(predictions)
```

```
Predicted marks:
[38.60851605 35.73403241]
```

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, predictions)

print("Mean Absolute Error:", round(mae, 2), "marks")
```

```
Mean Absolute Error: 0.67 marks
```

```
new_student = pd.DataFrame({
    "Study_Hours": [5],
    "Attendance": [75],
    "Previous_Marks": [60]
})

predicted_marks = model.predict(new_student)

print("Study Hours: 5")
print("Attendance: 75%")
print("Previous Marks: 60")
print("Predicted Final Marks:", round(predicted_marks[0], 2))
```

```
Study Hours: 5
Attendance: 75%
Previous Marks: 60
Predicted Final Marks: 60.03
```

```
comparison = pd.DataFrame({
    "Actual Marks": y_test.values,
    "Predicted Marks": predictions.round(2)
})

comparison["Absolute Error"] = (
    comparison["Actual Marks"] - comparison["Predicted Marks"]
).abs().round(2)

display(comparison)
```

	Actual Marks	Predicted Marks	Absolute Error
0	38	38.61	0.61
1	35	35.73	0.73

Next steps: Generate code with `comparison` New interactive sheet

```
import matplotlib.pyplot as plt

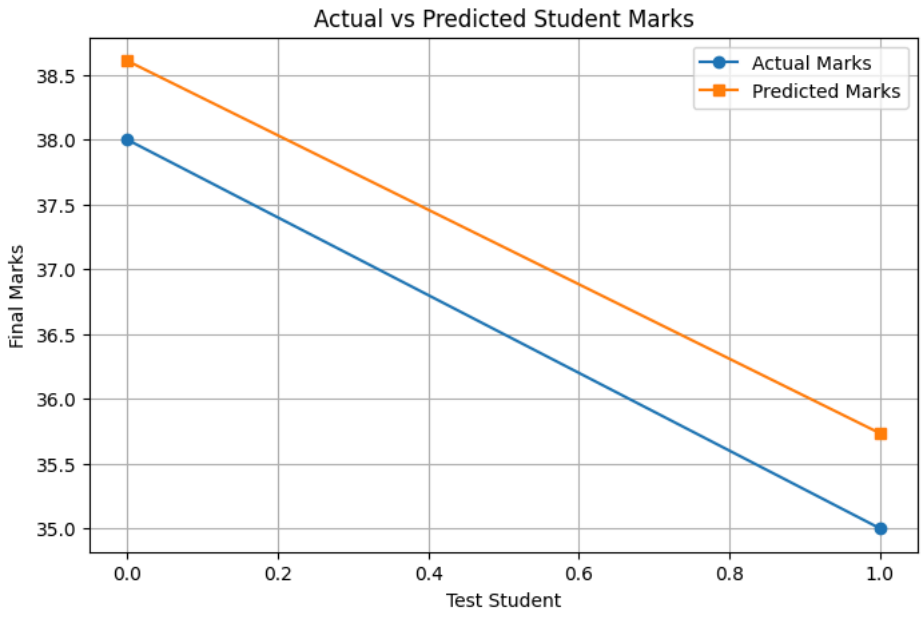
plt.figure(figsize=(8, 5))

plt.plot(
    range(len(y_test)),
    y_test.values,
    marker="o",
    label="Actual Marks"
)

plt.plot(
    range(len(predictions)),
    predictions,
    marker="s",
    label="Predicted Marks"
)

plt.xlabel("Test Student")
plt.ylabel("Final Marks")
plt.title("Actual vs Predicted Student Marks")
plt.legend()
plt.grid(True)

plt.show()
```




```
import joblib

joblib.dump(model, "student_performance_model.pkl")

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
import joblib
```

```
loaded_model = joblib.load("student_performance_model.pkl")

print("Saved model loaded successfully!")
```

```
Saved model loaded successfully!
```